Longest Common Substring

COMPLETE DOCUMENTATION (MEMOIZATION)

Problem Statement

Given two strings `s1` and `s2`, find the length of the longest common substring.

A substring consists of contiguous characters.

> **Example:**
>
> `s1 = "abcjklp"`
>
> `s2 = "acjkp"`
>
> **Longest Common Substring** = `"cjk"`
>
> **Length** = `3`

Important Difference

Longest Common Subsequence

Characters do not need to be adjacent.

```
abcde
ace

Answer = 3 ("ace")
```

Longest Common Substring

Characters must be continuous.

```
abcde
ace

Answer = 1
```

Only `"a"`, `"c"`, `"e"` are common substrings.

Step 1: Define the DP State

The most important part of this problem is understanding: What does `dp[i][j]` represent?

It does NOT mean:

Longest common substring in `s1[0...i]` and `s2[0...j]`

Instead:

`dp[i][j]` = Length of the longest common substring ending exactly at index `i` in `s1` and index `j` in `s2`.

Example

`s1 = abcde`

`s2 = xbcdz`

DP Table

```
      x  b  c  d  z
      0  1  2  3  4
   ---------------
a |   0  0  0  0  0
b |   0  1  0  0  0
c |   0  0  2  0  0
d |   0  0  0  3  0
e |   0  0  0  0  0
```

Observe:

- `dp[1][1]`
  `b == b`
  So: `dp[1][1] = 1`
  Substring ending at: `s1[1]='b'`, `s2[1]='b'` is `"b"`
- `dp[2][2]`
  `c == c`
  Therefore: `dp[2][2] = 1 + dp[1][1] = 2`
  Substring: `"bc"`
- `dp[3][3]`
  `d == d`
  Hence: `dp[3][3] = 1 + dp[2][2] = 3`
  Substring: `"bcd"`

Transition

If characters match:

$$dp[i][j] = 1 + dp[i-1][j-1]$$

because we extend the previous substring.

If characters don't match:

```
dp[i][j] = 0
```

because a substring is broken immediately.

Why do we need another variable ans?

Because `dp[i][j]` only stores: Longest common substring ending at `(i,j)`. The answer may end anywhere inside the table.

Example:
`s1 = abcde`
`s2 = xbcdz`

DP table:

```
      x b c d z
a     0 0 0 0 0
b     0 1 0 0 0
c     0 0 2 0 0
d     0 0 0 3 0
e     0 0 0 0 0
```

Notice: `dp[4][4] = 0` but the answer is `3` stored at `dp[3][3]`.

Therefore: `ans = max(ans, dp[i][j]);` must be maintained separately.

Recursive State Meaning

The function `solve(i,j)` returns: Length of the longest common substring ending exactly at `i` and `j`.

For example: `solve(3,3)` returns length of substring ending at `s1[3]` and `s2[3]`, not the overall answer.

Why do we call `solve(i-1,j)` and `solve(i,j-1)`?

These calls are not used to compute `dp[i][j]`.
Their purpose is only: **Visit every state in the DP table.**
Because the answer can occur anywhere.

Without them: `solve(n-1,m-1)` would only follow diagonal paths and many states would never be explored.

Flow Example

Suppose: `s1 = "abcde"`, `s2 = "xbcdz"`

Starting: `solve(4,4)`

First explore: `solve(3,4)` and `solve(4,3)`

These recursively visit: `(3,4)`, `(4,3)`, `(2,4)`, `(3,3)`, `(2,3)`, `(3,2)` ... Eventually every cell gets visited.

At `(1,1)` characters match: `b==b`. So: `dp[1][1]=1`, `ans=1`

At `(2,2)` characters match: `c==c`. Thus: `dp[2][2] = 1 + dp[1][1] = 2`. Update: `ans = max(1,2)`. Now `ans = 2`

At `(3,3)` characters match: `d==d`. Hence: `dp[3][3] = 1 + dp[2][2] = 3`. Update: `ans = max(2,3)`. Finally: `ans = 3`

Clean Memoization Code

```cpp
class Solution {
public:

    int ans = 0;
    vector<vector<int>> dp;

    int solve(int i, int j, string &s1, string &s2)
    {
        // Base case
        if (i < 0 || j < 0)
            return 0;

        // Already computed
        if (dp[i][j] != -1)
            return dp[i][j];

        // Visit all states
        solve(i - 1, j, s1, s2);
        solve(i, j - 1, s1, s2);

        // Length of common substring ending at i,j
        int curr = 0;

        if (s1[i] == s2[j])
        {
            curr = 1 + solve(i - 1, j - 1, s1, s2);

            // Update global answer
            ans = max(ans, curr);
        }

        return dp[i][j] = curr;
    }

    int longCommSubstr(string &s1, string &s2)
    {
        int n = s1.size();
        int m = s2.size();

        dp.assign(n, vector<int>(m, -1));

        solve(n - 1, m - 1, s1, s2);

        return ans;
    }
};
```

Mental Model

Think of every cell `dp[i][j]` as asking: *"If I force my substring to end exactly here, how long can it be?"*

Example:

```
      x b c d z
a     0 0 0 0 0
b     0 1 0 0 0
c     0 0 2 0 0
d     0 0 0 3 0
e     0 0 0 0 0
```

Each cell knows only about itself.

The variable `ans` acts like a camera watching all cells: `ans = max(ans, dp[i][j]);` and remembers the largest value encountered.

Summary

- **State:** `dp[i][j]` = Length of longest common substring ending exactly at `i,j`
- **Match:** `dp[i][j] = 1 + dp[i-1][j-1]`
- **Mismatch:** `dp[i][j] = 0`
- **Answer Location:** Not `dp[n-1][m-1]`. Instead: `max(dp[i][j])` which is maintained in `ans`

> This distinction: **"DP state stores local information, answer stores global information"** is one of the most important ideas in Dynamic Programming and appears again in problems like:
>
> - Maximum Path Sum
> - Diameter of Binary Tree
> - Maximum Sum BST
> - Longest ZigZag Path
> - Longest Increasing Path in Matrix
> - Maximum Product Subarray
> - Longest Common Substring
> - Maximum Rectangle in Binary Matrix